

C++ STL

```
#include <bits/stdc++.h>
using namespace std;

// 常用类型别名
typedef long long ll;
typedef long double ld;
typedef pair<int, int> PII;
typedef pair<ll, ll> PLL;

// 常用常量
const int INF = 1e9;
const ll LLINF = 1e18;
const int MOD = 1e9 + 7;
const double PI = acos(-1.0);
```

一、最核心容器：vector（动态数组）

1. 基础定义

```
vector<int> v;           // 空int数组
vector<int> v(10);       // 10个元素，默认值0
vector<int> v(10, 5);    // 10个元素，值都是5
vector<int> v = {1, 2, 3}; // 初始化列表
```

2. 核心函数

函数	作用	时间复杂度
<code>v.push_back(x)</code>	末尾添加元素x	O(1) 均摊
<code>v.emplace_back(x)</code>	末尾构造元素x (比push_back快)	O(1) 均摊
<code>v.pop_back()</code>	删除末尾元素	O(1)
<code>v.size()</code>	返回实际元素个数	O(1)
<code>v.empty()</code>	判断是否为空	O(1)
<code>v.clear()</code>	清空所有元素	O(n)
<code>v.resize(n)</code>	把大小改成n, 新增元素默认初始化	O(n)
<code>v.resize(n, val)</code>	把大小改成n, 新增元素值为val	O(n)
<code>v.reserve(n)</code>	预留n个元素的空间, 不改变size	O(n)
<code>v.erase(it)</code>	删除迭代器it指向的元素, 返回下一个元素的迭代器	O(n)
<code>v.erase(first, last)</code>	删除[first, last)区间的所有元素, 返回last	O(n)

3. 竞赛高频用法

```
// 1. 邻接表存图 (Dijkstra/BFS/DFS必用)
vector<vector<PII>> ver(n + 1);
ver[x].eb(y, w); // 加边x→y, 权值w

// 2. 遍历
for (int i = 0; i < v.size(); i++) {
    cout << v[i];
}
// 范围for (C++11+)
for (auto x : v) {
    cout << x;
}

// 3. 去重 (标准写法)
sort(all(v));
v.erase(unique(all(v)), v.end());

// 4. 循环删除元素 (唯一正确写法!)
for (auto it = v.begin(); it != v.end(); ) {
    if (*it == target) {
        it = v.erase(it); // erase返回下一个迭代器
    } else {
        it++;
    }
}
```

4. 易错点

- 下标访问必须保证 `i < v.size()` , 否则越界崩溃
- `reserve()` 只预留空间, 不能用下标访问, 必须用 `push_back` / `emplace_back`
- `vector` 可以直接用 `==` / `!=` 比较内容
- 迭代器失效: `erase`会导致被删除元素及之后的所有迭代器、引用全部失效

二、pair (键值对)

1. 基础定义

```
PII p;  
pair<int, string> p = {1, "abc"};
```

2. 成员访问

```
p.fi; // 第一个元素 (等价于p.first)  
p.se; // 第二个元素 (等价于p.second)
```

3. 竞赛高频用法

```
// 1. 存图的边 (终点, 权值)  
PII edge = {y, w};  
  
// 2. 优先队列存 (距离, 节点)  
priority_queue<PII> q;  
  
// 3. C++17结构化绑定 (推荐)  
for (auto [y, w] : ver[x]) {  
    // 直接用y和w  
}  
// 兼容写法  
for (auto &e : ver[x]) {  
    int y = e.fi;  
    int w = e.se;  
}
```

三、序列容器

1. string (字符串)

核心函数

```
string s = "abc";
s.size();      // 长度
s += "def";    // 拼接
s[0];          // 下标访问
s.substr(pos, len); // 从pos开始取len个字符
s.find("abc"); // 查找子串, 返回下标, 找不到返回string::npos
s.erase(pos);  // 删除从pos开始到末尾的所有字符
s.erase(pos, len); // 删除从pos开始的len个字符
s.erase(it);   // 删除迭代器it指向的字符, 返回下一个迭代器
s.erase(first, last); // 删除[first, last)区间的所有字符, 返回last
```

高频用法

```
// 删除字符串中所有的'a'
for (auto it = s.begin(); it != s.end(); ) {
    if (*it == 'a') {
        it = s.erase(it);
    } else {
        it++;
    }
}
```

易错点

- 迭代器失效：同vector，erase会导致被删除元素及之后的所有迭代器失效

2. deque（双端队列，竞赛高频）

支持两端O(1)增删，也支持随机访问

基础定义

```
deque<int> dq;          // 空双端队列
deque<int> dq(10);       // 10个元素, 默认值0
deque<int> dq(10, 5);    // 10个元素, 值都是5
deque<int> dq = {1, 2, 3}; // 初始化列表
```

核心函数

函数	作用	时间复杂度
<code>dq.push_front(x)</code>	开头添加元素x	$O(1)$
<code>dq.push_back(x)</code>	末尾添加元素x	$O(1)$
<code>dq.emplace_front(x)</code>	开头构造元素x（更快）	$O(1)$
<code>dq.emplace_back(x)</code>	末尾构造元素x（更快）	$O(1)$
<code>dq.pop_front()</code>	删除开头元素	$O(1)$
<code>dq.pop_back()</code>	删除末尾元素	$O(1)$
<code>dq.front()</code>	返回开头元素	$O(1)$
<code>dq.back()</code>	返回末尾元素	$O(1)$
<code>dq[i]</code>	随机访问第i个元素	$O(1)$
<code>dq.size()</code>	返回实际元素个数	$O(1)$
<code>dq.empty()</code>	判断是否为空	$O(1)$
<code>dq.clear()</code>	清空所有元素	$O(n)$
<code>dq.erase(it)</code>	删除迭代器it指向的元素，返回下一个迭代器	$O(n)$
<code>dq.erase(first, last)</code>	删除[first, last)区间的所有元素，返回last	$O(n)$

竞赛高频用法

① 普通BFS（比queue更灵活）

```
deque<int> dq;
vector<int> dis(n + 1, -1);

dq.push_back(1);
dis[1] = 0;

while (!dq.empty()) {
    int u = dq.front();
    dq.pop_front();

    for (auto [v, w] : ver[u]) {
        if (dis[v] == -1) {
            dis[v] = dis[u] + 1;
            dq.push_back(v);
        }
    }
}
```

② 0-1 BFS (边权只有0或1的最短路)

```
deque<PII> dq;
vector<int> dis(n + 1, INF);

dq.push_front({0, s});
dis[s] = 0;

while (!dq.empty()) {
    auto [d, u] = dq.front();
    dq.pop_front();

    if (d > dis[u]) continue;

    for (auto [v, w] : ver[u]) {
        if (dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            if (w == 0) {
                dq.push_front({dis[v], v}); // 0权边放队首
            } else {
                dq.push_back({dis[v], v}); // 1权边放队尾
            }
        }
    }
}
```

易错点

- 迭代器失效：erase会导致所有迭代器、引用全部失效（比vector更严格）

3. list（双向链表）

任意位置 $O(1)$ 插入删除，不支持随机访问

核心函数


```
list<int> l;
l.push_back(x); // 末尾添加
l.push_front(x); // 开头添加
l.pop_back(); // 删除末尾
l.pop_front(); // 删除开头
l.front(); // 开头元素
l.back(); // 末尾元素
l.size(); // 元素个数
l.empty(); // 判断空
l.clear(); // 清空
l.erase(it); // 删除迭代器指向的元素，返回下一个迭代器
l.erase(first, last); // 删除[first, last)区间的所有元素，返回last
```

易错点

- 不支持 `[]` 下标访问
- 迭代器失效：只有被删除元素的迭代器失效，其他所有迭代器、引用保持有效

四、容器适配器

1. priority_queue（优先队列/堆）

默认是大根堆（最大值在堆顶）

```
// 大根堆（默认）
priority_queue<int> q;
// 小根堆（Dijkstra必用！）
priority_queue<int, vector<int>, greater<int>> q;

// 存pair的小根堆（Dijkstra标准写法）
priority_queue<PII, vector<PII>, greater<PII>> q;
```

函数	作用
<code>q.push(x)</code>	插入元素x
<code>q.emplace(x)</code>	构造并插入x

<code>q.top()</code>	返回堆顶元素
<code>q.pop()</code>	删除堆顶元素
<code>q.empty()</code>	判断是否为空
<code>q.size()</code>	返回元素个数

⚠ 重要: `priority_queue` 没有 `erase` 函数, 只能删除堆顶元素。

2. queue (队列, BFS必用)

```
queue<int> q;
q.push(x);    // 队尾入队
q.front();    // 队首元素
q.back();     // 队尾元素
q.pop();      // 队首出队
q.empty();    // 判断空
q.size();     // 返回元素个数
```

重要: `queue` 没有 `erase` 函数, 只能删除队首元素。如果需要删除中间元素, 应该用 `deque` 代替。

3. stack (栈)

```
stack<int> s;
s.push(x);    // 栈顶入栈
s.top();      // 栈顶元素
s.pop();      // 栈顶出栈
s.empty();    // 判断空
s.size();     // 返回元素个数
```

**** 非常重要: `stack` 没有 `erase` 函数, 也没有迭代器****

- 只能删除栈顶元素 (用 `pop()`)
- 绝对不能删除栈中间的元素

- 如果需要随机删除, 应该用 `vector` 或 `deque` 模拟栈

五、有序关联容器

1. set (有序集合, 元素唯一)

核心函数

```
set<int> st;
st.insert(x);      // 插入元素x
st.count(x);       // 判断x是否存在 (返回0或1)
st.find(x);        // 查找x, 返回迭代器, 找不到返回st.end()
st.size();         // 元素个数
st.empty();        // 判断空
st.clear();        // 清空
st.erase(val);     // 删除所有值为val的元素, 返回删除的个数
st.erase(it);      // 删除迭代器it指向的元素, 返回下一个迭代器
st.erase(first, last); // 删除[first, last)区间的所有元素, 返回last
```

正确循环删除写法

```
for (auto it = st.begin(); it != st.end(); ) {
    if (*it == target) {
        it = st.erase(it);
    } else {
        it++;
    }
}
```

易错点

- 元素自动按升序排序
- 迭代器失效: 只有被删除元素的迭代器失效, 其他所有迭代器、引用保持有效

2. map (有序键值对, 键唯一)

核心函数

```
map<int, int> mp;
mp[key] = value; // 插入/修改
mp.count(key);   // 判断key是否存在 (返回0或1)
mp.find(key);    // 查找key, 返回迭代器, 找不到返回mp.end()
mp.size();       // 元素个数
mp.empty();      // 判断空
mp.clear();      // 清空
mp.erase(key);   // 删除所有键为key的元素, 返回删除的个数
mp.erase(it);    // 删除迭代器it指向的元素, 返回下一个迭代器
mp.erase(first, last); // 删除[first, last)区间的所有元素, 返回last
```

正确循环删除写法

```
for (auto it = mp.begin(); it != mp.end(); ) {
    if (it->se == target) { // 删除值为target的元素
        it = mp.erase(it);
    } else {
        it++;
    }
}
```

易错点

- 键自动按升序排序
- 迭代器失效: 同set

3. multiset / multimap (允许重复元素)

用法与set/map完全一致, 唯一区别是 `erase(val)` 会删除所有值为 `val` 的元素。如果只想删除一个值为 `val` 的元素:

```
auto it = st.find(val);
if (it != st.end()) {
    st.erase(it);
}
```

六、无序关联容器

1. unordered_set / unordered_map

核心函数

```
unordered_set<int> st;
st.insert(x);      // 插入元素x
st.count(x);       // 判断x是否存在
st.find(x);        // 查找x, 返回迭代器
st.size();         // 元素个数
st.empty();        // 判断空
st.clear();        // 清空
st.erase(val);     // 删除所有值为val的元素, 返回删除的个数
st.erase(it);      // 删除迭代器it指向的元素, 返回下一个迭代器
st.erase(first, last); // 删除[first, last)区间的所有元素, 返回last
```

易错点

- 元素无序，平均插入查找 $O(1)$
- 迭代器失效：只有被删除元素的迭代器失效，其他所有迭代器、引用保持有效

2. unordered_multiset / unordered_multimap

用法与unordered_set/unordered_map完全一致，`erase(val)` 同样会删除所有匹配元素。

七、算法库 <algorithm>

1. sort（排序， $O(n \log n)$ ）

```
// 默认升序
sort(all(v));
// 降序
sort(all(v), greater<int>());
// 自定义比较函数（贪心必用）
bool cmp(const int &a, const int &b) {
    return a > b; // 降序
}
sort(all(v), cmp);
```

2. 最值与交换

```
max(a, b);    // 最大值
min(a, b);    // 最小值
swap(a, b);   // 交换两个变量
*max_element(all(v)); // 数组最大值
*min_element(all(v)); // 数组最小值
```

3. 二分查找 ($O(\log n)$)

```
// 找第一个  $\geq x$  的位置
auto it = lower_bound(all(v), x);
int pos = it - v.begin();

// 找第一个  $> x$  的位置
auto it = upper_bound(all(v), x);
```

4. 去重

```
// 先排序，再去重
sort(all(v));
v.erase(unique(all(v)), v.end());
```

八、所有容器erase用法对比总表

容器类型	erase返回值	迭代器失效范围	时间复杂度
vector/deque/string	下一个迭代器	被删除元素及之后所有	O(n)
list	下一个迭代器	仅被删除元素	O(1)
set/map/multiset/multimap	下一个迭代器 (C++11+)	仅被删除元素	O(log n)
unordered_*	下一个迭代器 (C++11+)	仅被删除元素	平均 O(1)

九、通用erase最佳实践

1. 永远接收erase的返回值：这是所有容器通用的安全写法
2. 不要在erase后使用原来的迭代器：即使某些容器不会失效，接收返回值也是好习惯
3. 优先用迭代器erase：比按值erase更高效，且能精确控制删除哪个元素
4. multiset/multimap删除单个元素：必须先用 find() 找到迭代器再erase

十、竞赛高频组合用法

1. 邻接表存图（图论通用）

```
vector<vector<PII>> ver(n + 1);
for (int i = 0; i < m; i++) {
    int x, y, w;
    cin >> x >> y >> w;
    ver[x].eb(y, w);
    // 无向图加反向边
    // ver[y].eb(x, w);
}
```

2. 字母计数（滑动窗口/异位词必用）

```
vector<int> cnt(26, 0);  
cnt[c - 'a']++; // c是小写字母
```

3. Dijkstra小根堆

```
priority_queue<PII, vector<PII>, greater<PII>> q;  
q.emplace(0, s); // (距离, 节点)
```

十一、必记易错点

1. 整数除法：两个int相除会截断小数，必须转double： `1.0 * a / b`
2. 溢出：int最大值约2e9，超过用long long，无穷大统一用 `INF` (int) 或 `LLINF` (long long)
3. `priority_queue`默认大根堆：Dijkstra必须写 `greater<PII>`
4. `vector`越界：必须先 `resize()` 或 `push_back()` 才能用下标访问
5. C++17结构化绑定： `auto [y, w]` 不兼容老编译器，备用写法 `auto &e`

十二、竞赛必备：图论存图模板

0. 动态数组存图（vector 邻接表·竞赛最常用）

带权图（Dijkstra / 最短路 / 通用）


```
vector<vector<pair<int, int>>> g(n + 1); // 动态数组存图

for (int i = 0; i < m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    g[u].emplace_back(v, w); // 有向边 u → v
    g[v].emplace_back(u, w); // 无向图加反向边
}
```

无权图（BFS / DFS / 拓扑排序）

```
vector<vector<int>> g(n + 1); // 只存点，不存权值

for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);
}
```

1. 邻接表（最通用、最推荐）

无向图 / 有向图 标准版

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> PII;
const int INF = 1e9;

int main() {
    int n, m;
    cin >> n >> m;

    // 邻接表: ver[u] 存 (v, w)
    vector<vector<PII>> ver(n + 1);

    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        ver[u].emplace_back(v, w); // 有向边 u→v
        ver[v].emplace_back(u, w); // 无向图加反向边
    }

    // 遍历
    for (int u = 1; u <= n; u++) {
        for (auto [v, w] : ver[u]) {
            // 用 u, v, w
        }
    }
}
```

2. 邻接表（只存连通点，无边权）

适合 BFS / DFS / 拓扑排序

```
vector<vector<int>> g(n + 1);
for (int i=0; i<m; i++) {
    int u, v; cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);
}
```

3. 链式前向星（大数据/卡常数专用）

竞赛老题、大数据图必备

```
const int MAXN = 1e5 + 5;
const int MAXM = 2e5 + 5;

int head[MAXN], edge[MAXM], nxt[MAXM], val[MAXM], tot;

void init() {
    memset(head, -1, sizeof(head));
    tot = 0;
}

// 加边 u→v, 权值 w
void add(int u, int v, int w) {
    edge[tot] = v;
    val[tot] = w;
    nxt[tot] = head[u];
    head[u] = tot++;
}

// 遍历 u 的所有出边
for (int i = head[u]; ~i; i = nxt[i]) {
    int v = edge[i];
    int w = val[i];
}
```

4. 邻接矩阵 (适合小图、 $n \leq 1000$)

```
int n, m;
cin >> n >> m;
int g[1005][1005];

// 初始化
for (int i=1; i<=n; i++)
    for (int j=1; j<=n; j++)
        g[i][j] = INF;

for (int i=0; i<m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    g[u][v] = w;
    g[v][u] = w; // 无向图
}
```

5. 边结构体存图 (Kruskal / 差分约束 / 离线处理)

```
struct Edge {
    int u, v, w;
} e[M];

// 输入
for (int i=0; i<m; i++) {
    cin >> e[i].u >> e[i].v >> e[i].w;
}

// 排序 (最小生成树)
sort(e, e+m, [](Edge a, Edge b) {
    return a.w < b.w;
});
```

6.0-1 BFS 专用存图 (边权仅 0/1)

```
vector<vector<PII>> g(n+1);
for (int i=0; i<m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    g[u].emplace_back(v, w);
    g[v].emplace_back(u, w);
}
```

7. 带边编号存图 (处理边修改、边查询)

```
struct Edge {
    int to, w, id;
};
vector<vector<Edge>> g(n+1);

for (int i=0; i<m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    g[u].push_back({v, w, i+1});
    g[v].push_back({u, w, i+1});
}
```

8. 树上倍增 / LCA 专用存图

```
vector<vector<int>> g(n+1);
vector<vector<int>> fa(n+1, vector<int>(20));
vector<int> dep(n+1);

// 加边
g[u].push_back(v);
g[v].push_back(u);
```

